

Multi Threading & Asynchronous Programming – Part 1

NET 022 - Advanced C# Programming

Prepared By: Supun Kandaudahewa
supun.kandaudahewa@humber.ca

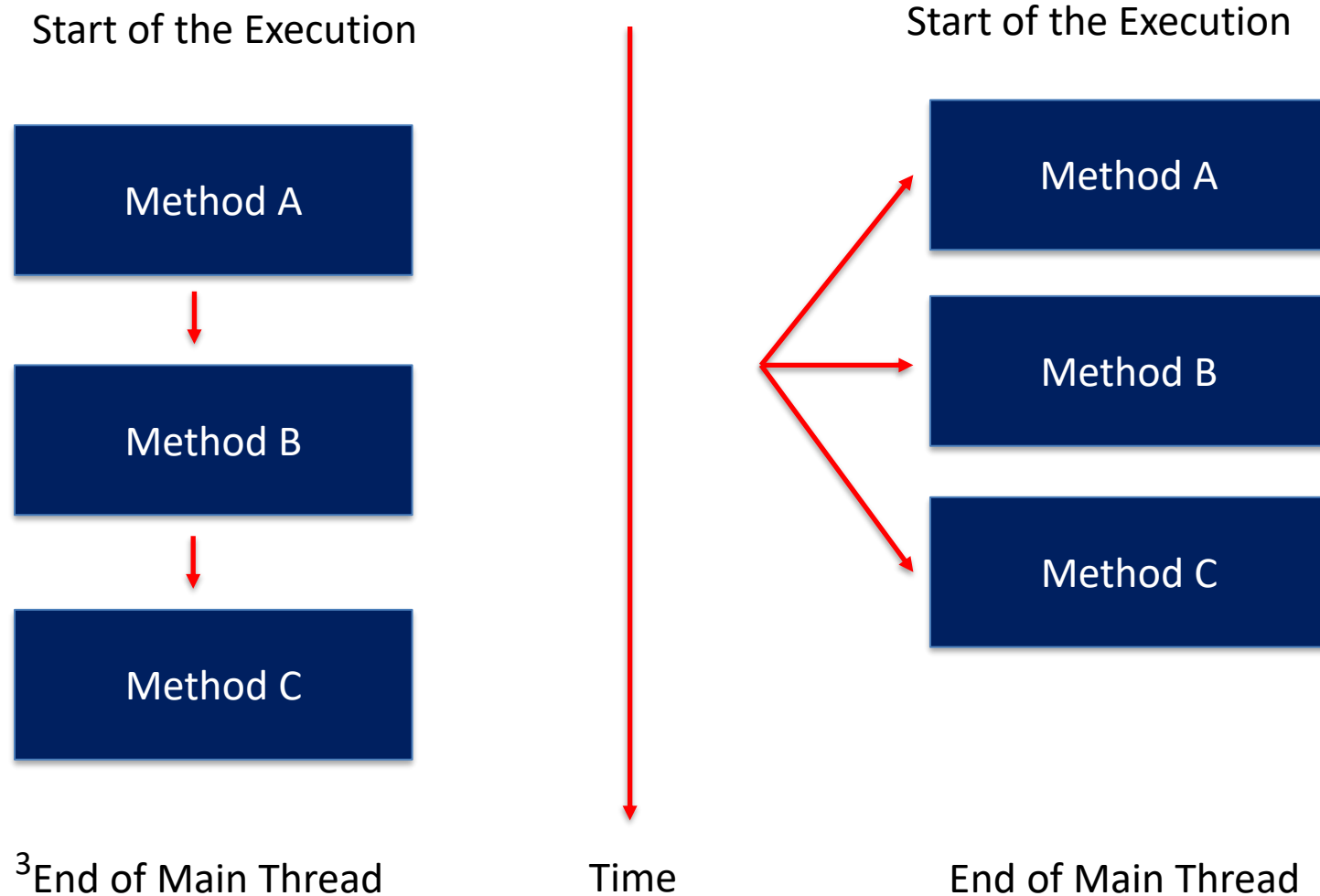


**WE ARE
HUMBER**

Agenda

- Process Vs Thread
- Thread Pool and Thread Class
- Foreground Vs Background Threads
- Thread Priority
- Race Conditions and Deadlocks
- Thread Synchronization Techniques
- Tasks
- Tasks Continuation and Cancellation
- Task and Data Parallelism

The Order of Execution



Process

- **Process** is an operating system–level concept used to describe all the resources used by a running application.
- For each application loaded into memory the OS:
 - Creates a separate and isolated process for use during its lifetime.
 - Defines a virtual address space.
- Libraries of code are mapped into the address space.

Threads

Threads execute code.

- A path of execution through all code within a single process.
- Have access to all data within that process.
- Each thread has its own stack and its own copy of CPU registers as well as Thread Local Storage (TLS)
- A process with no foreground threads exits – it can no longer perform work.
- A thread ends when the delegate passed to the Thread's constructor finishes executing.

Threads – Use cases

- Opportunity to scale by parallelizing CPU-bound operations.
 - Assuming multi-core / multi-processor hardware.
 - Threading uses time slices to give the "illusion" that multiple things are happening at the same time on a single core.
- Perform CPU-bound work while I/O operations are waiting.
 - Performing other actions until a large dataset is loaded etc.
- Maintain a responsive UI.
 - Off load lengthy operations to a separate thread.
 - Using thread priorities to ensure user critical threads have priority.

Multi Threading Caveats – Does threads always perform better?

- Slower execution time on single-core machines for CPU bound code.
- Using more threads does NOT mean faster execution.
- Added program complexity:
 - Lines of code is increased.
 - Readability decreased.
 - Maintainability becomes harder.
 - Testability becomes complex.
- Altering thread setting can negatively impact on the performance of entire system.

Thread Pools

- Provides a pool of threads that can be used to execute tasks, post work items, process asynchronous I/O, wait on behalf of other threads, and process timers.
- Lives in System.Threading namespace.
- All the available threads are managed by the ThreadPool class.
- Let's see an example.

[ThreadPool Class \(System.Threading\) | Microsoft Learn](#)

Thread Class

Creates and controls a thread, sets its priority, and gets its status.

- The **Thread** class was the original mechanism used to start a thread in the .Net Framework.
- With the **Thread** class, you can create and control threads better than using the **ThreadPool** or the latest **async/await**.
- But the **creation and destruction of threads incur high resource utilization**, mainly CPU overhead.
- Let's create a thread and see for our selves.

[Thread Class \(System.Threading\) | Microsoft Learn](#)

Foreground vs Background Threads

- The process of the application keeps running as long as at least one foreground thread is running.
- If more than one foreground thread is running and the Main method ends, the process of the application remains active until all foreground threads finish their work.
- A thread you create with the **Thread class, by default, is a foreground thread.**
- **Thread pool** threads are always **background threads.**

Thread Priority

- You can influence the base priority of the thread by setting the **Priority** property.
- The **Priority** property requires a value that is defined by the **ThreadPriority** enumeration. The levels defined
 - Highest
 - AboveNormal
 - Normal
 - BelowNormal
 - Lowest
- Setting priority property provides **only a suggestion to the scheduler** about what to do and does not guarantee results.

Controlling Threads

- A thread is created by invoking the Start method.
- After invoking start a thread is still **not** in a running state, but in the **Unstarted** state.
- A thread changes to a **Running** state when the thread scheduler selects the thread to run.
- You can use the Thread.ThreadState property to find the current state of a thread.

Race Conditions

- Occur if two or more threads access the same objects.
- Let's see an example.
- Looking at the code it might seem that the result at the end will be 650 -> $50 + 100 - 500 + 1000$.
- But if you try executing the code you will find out that it is not always true.

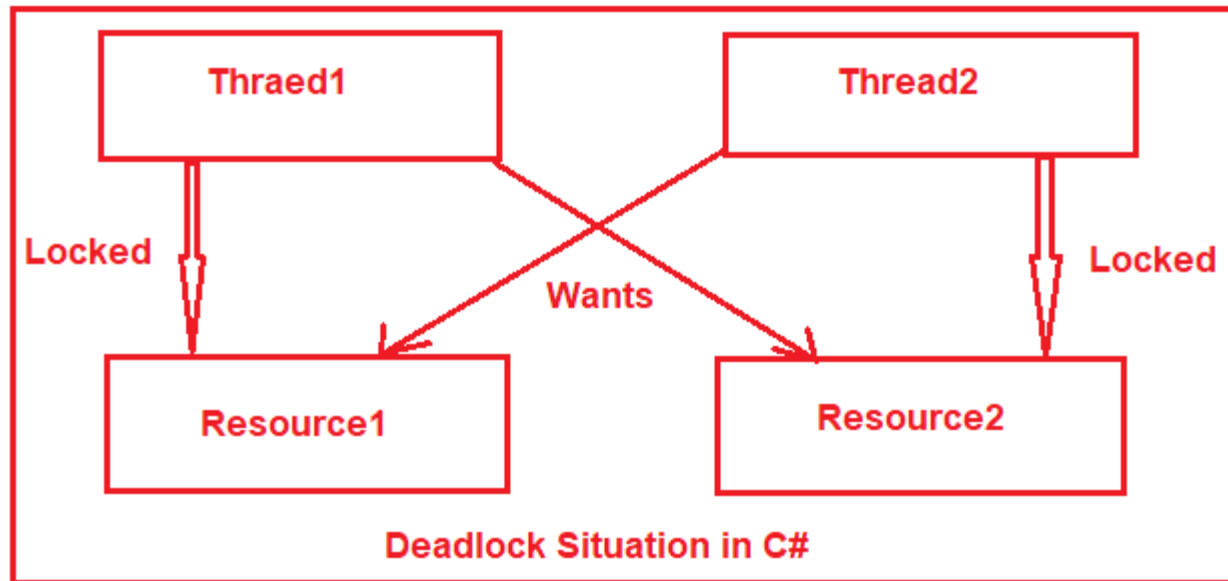
```
Starting Transaction1  
Starting Transaction2  
Starting Transaction3  
The balance: 650
```

```
Starting Transaction1  
Starting Transaction2  
Starting Transaction3  
The balance: -350
```

Note: Please open the sample solution provided in the class folder to follow along.

Deadlocks

- When two processes are waiting for each other directly or indirectly, it is called deadlock.



Thread Synchronization Techniques

- **Lock** – The simplest way to do it.
- **Monitor** – We can use exception handling
- **Manual Reset Event** – One thread is doing the work and other thread needs to wait for the first thread to finish
- **Auto Reset Event** – Need to pass the flag, or else the thread would not start.
- **Mutex** - Each calling thread is blocked until it acquires ownership of the mutex
- **Semaphore** – Not a blocking mechanism but a signaling system.

Note: Please open the sample solution provided in the class folder to follow along.

Task

- The **Tasks** class resides in the System.Threading.Tasks namespace.
- A Task represents an asynchronous operation.
- Task does not return a value by default → Task<TResult> class can be used if result is needed.
- Executes asynchronously on a thread pool thread.
- **TaskStatus** enum will help us to determine the status of the Task
 - Cancelled
 - Created
 - Faulted
 - RanToCompletion
 - Running
 - WaitingForActivation
 - WaitingForChildrenToComplete
 - WaitingToRun

Task Instantiation

- Let's see how we can **Instantiate** and **Run** a task:
 - Using Constructor
 - Using TaskFactory
 - Using Task.Run
 - Using Task.RunSynchronously

Note: Please open the sample solution provided in the class folder to follow along.

Task Results

- When a task is finished, it can write some stateful information to a shared object.
- The generic class **Task<TResult>** can be used for this purpose.
- The type parameter **TResult** specifies the type of the result written into the **Result** property of the task.
- An alternative, and the most common way to start a task in the .NET Framework 4, is to call the static **TaskFactory.StartNew** or **TaskFactory<TResult>.StartNew**.

Note: Please open the sample solution provided in the class folder to follow along.

<https://learn.microsoft.com/en-us/dotnet/api/system.threading.tasks.task-1?view=net-6.0>

Task Continuation

- In asynchronous programming, it's common for one asynchronous operation to invoke a second operation on completion.
- Continuations allow descendant operations to consume the results of the first operation.
- A continuation task is an asynchronous task that's invoked by another task, known as the antecedent. When the antecedent finishes it will automatically invoke the next task.

Note: Please open the sample solution provided in the class folder to follow along.

[Chaining tasks using continuation tasks | Microsoft Learn](#)

Task Cancellation

- The cancellation framework enables the canceling of long-running tasks in a standard manner.
- We use **CancellationTokenSource** to achieve this objective.
- Every blocking call should support this mechanism.
- The cancellation framework is based on **cooperative behavior; it is not forceful**.
- A long-running task checks whether it is canceled and returns control accordingly.

Note: Please open the sample solution provided in the class folder to follow along.

Task and Data Parallelism

- There are two types of parallelism.
 - Task parallelism, code that's using the CPU is parallelized
 - Data parallelism, data collections are used
- The **Parallel** class dynamically distributes workload across available CPUs and handles all low-level details of thread management **while shielding you from all the complexity.**
- Defines static methods like:
 - Parallel.For
 - Parallel.ForEach
 - Parallel.Invoke

Parallel.For Method

- Similar to the **for-loop** statement but the iterations run in parallel.
- The order of iterations is not defined.
- Takes 3 parameters
 - First two parameters define the start and end of the loop.
 - The third parameter is a delegate.

Note: Please open the sample solution provided in the class folder to follow along.

Parallel.ForEach Method

- Iterates through a collection implementing **IEnumerable**.
- If you need to break up the loop, you can use an overload of the ForEach method with a ParallelLoopState parameter.

Note: Please open the sample solution provided in the class folder to follow along.

Parallel.Invoke Method

- If multiple tasks should run in parallel, you can use the **Parallel.Invoke** method.
- Allows the passing of an array of Action delegates.

Note: Please open the sample solution provided in the class folder to follow along.

THANK YOU.



WE ARE

HUMBER